

Informe de Validación Estratégica: Viabilidad de Mercado y Arquitectura Técnica de "Thalamus" para la Economía Agéntica

Fecha: 16 de Enero de 2026

Para: Comité Ejecutivo de Estrategia de Producto e Infraestructura

De: Analista Senior de Infraestructura de Software y Estratega de Mercado Cloud

Asunto: Análisis de Viabilidad, Fricción Técnica y Ventaja Competitivo del Servidor de Identidad Thalamus (Elixir/BEAM) ante el Colapso de los IdP Tradicionales en Cargas de Trabajo de IA Agéntica.

1. Resumen Ejecutivo y Tesis de Inversión

El presente documento constituye un informe exhaustivo de investigación y validación de mercado, encargado para evaluar la viabilidad técnica y comercial del proyecto "Thalamus", un Proveedor de Identidad (IdP) basado en la máquina virtual BEAM (Erlang/Elixir). La investigación, concluida en enero de 2026, confirma de manera contundente la tesis central: **Los proveedores de identidad incumbentes (Auth0/Okta, Clerk, Keycloak) enfrentan una obsolescencia estructural ante las demandas de la naciente "Economía Agéntica".**

Nuestra validación se fundamenta en tres pilares críticos que definen una ruptura en el mercado:

- 1. Colapso del Modelo Económico (El "Impuesto IdP"):** Los modelos de precios actuales, diseñados para la autenticación humana (MAU - Monthly Active Users), son depredadores cuando se aplican a la autenticación Máquina a Máquina (M2M) de alta frecuencia. El análisis forense de precios revela que una flota de agentes generando **10 millones de tokens mensuales** incurriría en costos operativos de entre **\$10,000 USD (Clerk)** y **\$50,000 USD (Stytch)** mensuales. En contraste, la arquitectura propuesta para Thalamus, operando sobre hardware AWS Graviton (ARM), puede sostener esa misma carga por aproximadamente **\$333 USD mensuales**, representando una reducción de costos superior al **99%**.
- 2. Fricción Técnica y Latencia de Cola (El Muro de Java/Node):** La infraestructura legada basada en la JVM (Java Virtual Machine) y el Event Loop de Node.js es incapaz de garantizar la latencia determinista requerida por los enjambres de agentes (Agent Swarms). Los eventos de "Stop-the-World" del Garbage Collector en plataformas como Keycloak introducen picos de latencia ($p_{99} > 500\text{ms}$) que rompen las cadenas de razonamiento (Chain-of-Thought) de los LLMs, incrementando los costos de inferencia y degradando la experiencia de usuario. La arquitectura de "share-nothing" de Elixir/BEAM

ofrece un foso técnico validado para mantener latencias de sub-milisegundos bajo concurrencia masiva.

3. **Vacío de Seguridad en Protocolos Emergentes:** La adopción explosiva del **Model Context Protocol (MCP)** de Anthropic ha creado vectores de ataque críticos, como el problema del "Confused Deputy" y el robo de tokens en servidores locales. El mercado carece de un orquestador de identidad que implemente nativamente las especificaciones de seguridad emergentes, como el borrador IETF "AAuth" (Agentic Authorization), dejando a las empresas vulnerables o dependientes de soluciones parcheadas.

Recomendación: Se emite una calificación de **"GO" (Proceder)** para el desarrollo de Thalamus. Existe una ventana de oportunidad estratégica de 18 a 24 meses para capturar el mercado de infraestructura de identidad para IA, posicionando a Thalamus no como un competidor genérico de Auth0, sino como el motor de identidad de baja latencia y alta frecuencia indispensable para la orquestación de agentes autónomos.

2. Contexto Macro-Estratégico: El Ascenso de la Identidad No Humana (NHI)

Para comprender la magnitud de la oportunidad de Thalamus, es imperativo analizar primero el cambio tectónico en la naturaleza de la identidad digital que se ha cristalizado hacia enero de 2026. Hemos transitado de una era de Identidad Centrada en el Humano (HCI) a una era dominada por la Identidad No Humana (NHI), impulsada por la proliferación de agentes autónomos de Inteligencia Artificial.

2.1. De la Automatización Estática a la Autonomía Agéntica

Históricamente, la identidad "máquina" se limitaba a cuentas de servicio estáticas para pipelines de CI/CD o trabajos cron, caracterizadas por credenciales de larga duración y comportamientos predecibles. En contraste, la Economía Agéntica introduce actores no deterministas. Un agente de IA, impulsado por un LLM, decide en tiempo de ejecución qué herramientas invocar, qué bases de datos consultar y con qué otros agentes colaborar.

Esta naturaleza dinámica ha roto el paradigma de seguridad tradicional. Un agente no puede operar con una clave API estática con permisos de "superusuario" (God-mode), ya que esto amplifica catastróficamente el riesgo de inyección de prompt (Prompt Injection). La seguridad moderna exige que los agentes realicen una autenticación continua, solicitando tokens efímeros con el mínimo privilegio necesario (Least Privilege) para cada tarea específica. Esto transforma el patrón de tráfico de autenticación: de unos pocos inicios de sesión humanos al día, pasamos a miles de solicitudes de tokens por minuto por cada agente activo.

2.2. El Protocolo de Contexto del Modelo (MCP) y la Crisis de

Seguridad

La industria ha estandarizado la conectividad de datos para LLMs a través del **Model Context Protocol (MCP)**, promovido por Anthropic. Si bien MCP resuelve el problema de la interoperabilidad de datos, ha introducido, paradójicamente, una superficie de ataque masiva y mal protegida.

2.2.1. El Problema del "Confused Deputy" y la Centralización de Riesgo

Nuestra investigación sobre las vulnerabilidades de MCP revela que los servidores MCP actúan como agregadores de alto valor, almacenando tokens de acceso para múltiples servicios críticos (Gmail, Slack, Google Drive, GitHub) en un solo punto.¹

- **Vector de Ataque:** Atacantes pueden explotar servidores MCP proxy para crear vulnerabilidades de "Confused Deputy" (Diputado Confundido). Esto permite a clientes maliciosos obtener códigos de autorización sin el consentimiento adecuado del usuario, aprovechando la combinación de Client IDs estáticos y registros dinámicos inseguros.²
- **Riesgo de Inyección:** Dado que el asistente de IA interpreta comandos en lenguaje natural antes de enviarlos al servidor MCP, existe un vector de ataque mediante inyecciones de prompt indirectas. Un correo electrónico aparentemente inofensivo podría contener instrucciones ocultas que, al ser procesadas por el agente, instruyan al servidor MCP a exfiltrar documentos financieros a un servidor externo.¹

2.2.2. La Insuficiencia de las Soluciones Actuales

Las especificaciones actuales de MCP recomiendan el uso de OAuth 2.1 y el Registro Dinámico de Clientes (RFC 7591) para mitigar estos riesgos.³ Sin embargo, la implementación práctica de estos estándares en la infraestructura existente es deficiente:

- **Fricción de Registro:** Los IdP tradicionales como Auth0 no están optimizados para el registro dinámico y efímero de miles de clientes agentes.
- **Falta de Visibilidad:** Herramientas de seguridad emergentes como **Oasis Security** y **Entro Security** han surgido para *monitorear* y *descubrir* estas identidades no humanas⁴, pero no resuelven el problema de la *emisión* performante y segura de credenciales. Actúan como cámaras de vigilancia, pero Thalamus debe ser la bóveda segura.

2.3. La Emergencia del Estándar "AAuth" (Agentic Authorization)

Un desarrollo crítico en el panorama de estandarización es el borrador IETF para **"AAuth"**, una extensión de OAuth 2.1 diseñada específicamente para agentes confidenciales.⁶

- **El Mecanismo:** A diferencia del flujo de código de dispositivo (RFC 8628), AAuth introduce descripciones en lenguaje natural y parámetros de "razón" (reason) proporcionados por el agente, optimizados para la entrega de tokens mediante mecanismos asíncronos como HTTP Polling, Server-Sent Events (SSE) o WebSockets.⁷
- **Oportunidad para Thalamus:** Los incumbentes son lentos en adoptar borradores

experimentales ("drafts"). Thalamus tiene la oportunidad de convertirse en la **implementación de referencia** para OAuth, capturando a los desarrolladores que buscan cumplimiento normativo y seguridad "by-design" para sus flotas de agentes.

3. Análisis de Fricción Técnica: El Dolor del Desarrollador en 2026

La validación de Thalamus no es solo económica, sino fundamentalmente física: se trata de las limitaciones físicas de las arquitecturas de software actuales ante la concurrencia masiva. El análisis de foros de ingeniería, issues de GitHub y discusiones en Reddit revela una frustración creciente con las limitaciones de las pilas tecnológicas basadas en Java (Keycloak) y Node.js (Auth0 Extensibility).

3.1. La Tiranía del "Stop-the-World": Cuellos de Botella en la JVM

Keycloak sigue siendo el estándar de facto para la gestión de identidad open-source. Sin embargo, su arquitectura basada en Java Virtual Machine (JVM) presenta limitaciones severas para cargas de trabajo M2M de alta frecuencia.

3.1.1. Latencia de Garbage Collection (GC)

En un entorno agéntico, la latencia es costo. Un agente que espera 500ms por un token es un agente que consume tiempo de GPU inactiva y degrada la experiencia del usuario.

- **El Fenómeno:** Bajo alta carga de autenticación, la JVM genera millones de objetos efímeros (solicitudes, tokens, sesiones). Cuando el espacio de memoria (Heap) se llena, el Garbage Collector se activa. A pesar de las mejoras en recolectores modernos como ZGC o Shenandoah, en escenarios de alta presión de memoria, se producen eventos de "Stop-the-World" donde **todos los hilos de la aplicación se pausan** para permitir la limpieza de memoria.⁹
- **Impacto en el Mundo Real:** Pruebas de rendimiento indican que Keycloak sufre degradación severa bajo latencia de red, con tiempos de respuesta que se disparan a cientos de milisegundos.¹⁰ Ingenieros reportan que para manejar 500,000 logins por segundo (una carga plausible para una flota grande de IoT o agentes), se requeriría una infraestructura monstruosa de ~1,700 servidores, lo cual es inviable.¹¹
- **Configuración Compleja:** La mitigación de estos problemas requiere un ajuste ("tuning") esotérico de los parámetros de la JVM (tamaños de Heap, Metaspace, flags de GC), lo que añade una carga operativa significativa a los equipos de DevOps.¹²

3.1.2. Agotamiento de Pools de Conexiones y Hilos

El modelo de concurrencia de Java mapea hilos de la aplicación a hilos del sistema operativo (1:1 o M:N con Project Loom, aunque este último aún enfrenta madurez en ecosistemas

legados).

- **El Problema de la Manada en Estampida (Thundering Herd):** Cuando miles de agentes solicitan renovación de tokens simultáneamente, Keycloak agota rápidamente su pool de conexiones a la base de datos.¹⁴ Se observan errores de "Too many connections" incluso con configuraciones de pool agresivas, debido a que las transacciones mantienen las conexiones abiertas más tiempo del necesario.
- **Bloqueo de Hilos:** Las sondas de salud (Health Probes) y las operaciones de base de datos bloqueantes pueden saturar el pool de hilos de ejecución (Executor Pool) de Quarkus/Keycloak, llevando a que el servicio deje de responder (Load Shedding) y devuelva errores 503, causando fallos en cascada en los agentes dependientes.¹⁵

3.2. Node.js y el Bloqueo del Event Loop (El Caso Auth0)

Auth0 permite extensibilidad mediante "Actions" y "Rules", que se ejecutan en un entorno Node.js.

- **Limitación Arquitectónica:** Node.js es monohilo. Si bien es excelente para I/O asíncrono, cualquier operación intensiva en CPU —como la firma criptográfica de miles de JWTs (JSON Web Tokens)— bloquea el Event Loop principal.
- **Consecuencia:** En un enjambre de agentes, una ráfaga de solicitudes de tokens que requieran firma criptográfica degradará la latencia para *todas* las demás solicitudes concurrentes, introduciendo una variabilidad de latencia (Jitter) inaceptable para sistemas de tiempo real.

4. Validación del Foso Técnico Elixir/BEAM

Ante las deficiencias de Java y Node.js, nuestra investigación valida que la elección de Elixir y la máquina virtual BEAM (Erlang) no es una preferencia estética, sino una ventaja competitiva estructural y defendible (un "Moat").

4.1. Aislamiento de Procesos: La Solución a la Latencia de Cola

La diferencia fundamental radica en el modelo de concurrencia.

- **Sin Memoria Compartida:** En BEAM, cada petición de autenticación se ejecuta en su propio "Proceso Ligero" (Lightweight Process), que tiene su propio Heap de memoria y recolector de basura independiente.¹⁶
- **Recolección de Basura Per-Proceso:** A diferencia de Java, donde el GC pausa todo el sistema, en Elixir, cuando un proceso termina (e.g., se emite el token), su memoria se libera instantáneamente sin afectar a los otros millones de procesos concurrentes. Esto garantiza que la actividad de un agente ruidoso **nunca** impacte la latencia de otro agente.
- **Planificación Preemptiva:** El planificador de BEAM asigna un número estricto de

"reducciones" (operaciones) a cada proceso antes de pausarlo forzosamente para dar paso a otros. Esto asegura un sistema de "tiempo real blando" donde la latencia es predecible y consistente, eliminando los picos de p99 que plagan a la JVM.¹⁷

4.2. ETS (Erlang Term Storage) vs. La Dependencia de Redis

La mayoría de las arquitecturas de identidad modernas requieren una capa de caché externa (Redis) para gestionar sesiones y tokens efímeros con rapidez. Esto introduce latencia de red (serialización/deserialización) y complejidad operativa.

- **El Arma Secreta (ETS):** Thalamus puede aprovechar ETS, una base de datos en memoria integrada directamente en la VM. Benchmarks comparativos muestran que ETS es significativamente más rápido que Redis para accesos locales. Un benchmark específico muestra que ETS puede ser **6 a 10 veces más rápido** en operaciones de lectura/escritura concurrente al evitar el stack de red TCP y la sobrecarga de serialización JSON necesaria para comunicarse con Redis.¹⁸
- **Simplicidad Operativa:** Al eliminar la necesidad de un clúster de Redis gestionado (como AWS ElastiCache), Thalamus reduce drásticamente el Costo Total de Propiedad (TCO) y los puntos de fallo del sistema.¹⁹

4.3. Benchmarks de Realidad vs. Vanidad

Es común encontrar benchmarks sintéticos ("Hello World") donde frameworks de Java o Rust superan a Phoenix en throughput bruto (RPS). Sin embargo, en el contexto de identidad, la métrica crítica es la **estabilidad bajo carga**.

- **Consistencia:** Reportes de ingeniería destacan que mientras Java puede mostrar un alto throughput promedio, sus latencias de cola (tail latency) son erráticas. Phoenix/Elixir, aunque pueda tener un techo teórico de RPS más bajo en un solo núcleo, mantiene una latencia plana y predecible hasta el punto de saturación, lo cual es vital para la orquestación de agentes que dependen de tiempos de respuesta deterministas para no romper sus ventanas de contexto o timeouts.²¹

5. Panorama Competitivo: El Océano Rojo de la Identidad

El mercado de IAM (Identity and Access Management) está saturado, pero la mayoría de los jugadores están optimizados para casos de uso B2C/B2B humanos, dejando el segmento M2M/Agéntico mal atendido.

5.1. Los Incumbentes: Auth0 (Okta) y Clerk

- **Auth0 (Okta):** El líder indiscutible, pero atrapado en su propio éxito. Su modelo de negocio está orientado a la venta empresarial con ciclos largos. Su arquitectura "legacy"

y su estructura de precios (ver sección 6) son hostiles para startups de IA ágiles.

- **Clerk:** Ha ganado tracción masiva en el ecosistema React/Next.js por su excelente DX (Developer Experience). Sin embargo, su reciente incursión en M2M (GA en Octubre 2025) revela una debilidad estratégica: su backend no está optimizado para la escala masiva de tokens efímeros, lo que se refleja en un modelo de precios punitivo por creación de token.²³

5.2. Los Retadores de Nicho: Stytle y FusionAuth

- **Stytle:** Se posiciona como una alternativa "API-first". Aunque ofrece una capa gratuita generosa para usuarios, su precio para tokens M2M escala linealmente de manera agresiva (\$0.005/token), lo que lo descarta para altos volúmenes.²⁴
- **FusionAuth:** Su propuesta de valor de "self-hosting" es la amenaza más directa para Thalamus. Permite despliegue en infraestructura propia, eliminando el costo por usuario activo en ciertos planes. Sin embargo, su núcleo es Java, heredando los problemas de GC y consumo de recursos mencionados anteriormente. Además, sus características M2M "ilimitadas" están cerradas tras licencias Enterprise costosas.²⁶

5.3. El Ecosistema de Seguridad NHL: ¿Competidores o Socios?

Empresas emergentes como **Entro Security** y **Oasis Security** están recibiendo mucha atención e inversión.⁴

- **Posicionamiento:** Estas empresas se enfocan en la *observabilidad*, el *descubrimiento* de secretos y la gestión de postura (CSPM/DSPM) para identidades no humanas. Detectan cuando un token se fuga o se usa anómalamente.
- **La Brecha:** No son emisores de identidad (Issuers). Dependen de que exista un IdP subyacente. Thalamus no compite directamente con ellos, sino que complementa su oferta. Thalamus es la fábrica de tokens; Entro es el auditor de seguridad. Una integración nativa con estas plataformas sería un diferenciador clave.

6. Análisis Forense de Precios: El "Impuesto IdP"

El argumento más letal a favor de Thalamus es la insostenibilidad económica de los proveedores actuales para la economía agéntica. A continuación, presentamos un desglose financiero detallado.

Escenario de Modelado:

Una empresa mediana de IA opera una flota de 1,000 agentes autónomos.

- Cada agente realiza tareas complejas que requieren re-autenticación o tokens de alcance específico cada 5 minutos (para seguridad de "Least Privilege").
- Actividad diaria por agente: ~100 tokens.
- Volumen Mensual Total: **10,000,000 (Diez Millones) de Tokens M2M.**

6.1. Tabla Comparativa de Costos Mensuales (Proyección Enero 2026)

Proveedor	Modelo de Precios	Costo Unitario (M2M)	Costo Mensual (10M Tokens)	Análisis de Viabilidad
Auth0 (Okta)	Tiered / Add-on	Opaco / Negociado	>\$25,000 USD*	El plan "Essentials" incluye solo 1,000 tokens/mes. El exceso requiere contratos "Enterprise" con precios opacos y mínimos anuales altos. ²⁹
Clerk	Usage Based	\$0.001 (Creación)	\$10,000 USD	Cobra por <i>creación</i> de token. Incentiva malas prácticas de seguridad (TTLs largos) para ahorrar dinero. La verificación de tokens opacos añade \$0.0001 extra. ²³
Stytch	Usage Based	\$0.005 / Token	\$50,000 USD	Incluye 1,000 tokens gratis. El exceso se cobra a una

				tarifa plana de medio centavo, lo cual es prohibitivo a escala. ²⁴
FusionAuth	Licencia / Hosting	Incluido (Enterprise)	~\$3,300 - \$5,600 USD	Requiere plan "Enterprise" para entidades ilimitadas. Aunque el software permite self-hosting, la licencia es costosa. ²⁷
Thalamus	Self-Hosted (AWS)	Costo de Cómputo	~\$333 USD	Basado en infraestructura AWS Graviton (c7g.2xlarge + RDS). Sin costo de licencia por token.

6.2. Desglose de Costos de Infraestructura Thalamus (AWS)

Para soportar 10 millones de tokens mensuales con la eficiencia de BEAM, no se requiere un cluster masivo.

- **Servidor de Aplicación:** Instancia AWS EC2 c7g.2xlarge (Graviton3, ARM64).
 - Especificaciones: 8 vCPU, 16 GB RAM.
 - Costo Mensual (On-Demand): ~\$211.70 USD.³³
 - *Nota:* Gracias a la eficiencia de Elixir, esta instancia probablemente esté sobredimensionada, pero ofrece margen de seguridad para picos.
- **Base de Datos:** Amazon RDS para PostgreSQL db.m7g.large.
 - Especificaciones: 2 vCPU, 8 GB RAM.
 - Costo Mensual: ~\$122.64 USD.³⁴
- **Caché: \$0 USD.** (Uso de ETS en el nodo de aplicación, eliminando ElastiCache Redis).
- **Total Operativo: ~\$334.34 USD al mes.**

Conclusión Financiera: Thalamus ofrece una **reducción de costos del 96.6%** frente a la

opción comercial más cercana (Clerk) y del **99.3%** frente a opciones como Styth. Este margen es el "Impuesto IdP" que las empresas están desesperadas por eliminar.

7. Recomendaciones Estratégicas y Hoja de Ruta

Basado en la evidencia técnica y económica, la oportunidad para Thalamus es clara pero requiere una ejecución precisa que evite la competencia frontal por la identidad humana (login de usuarios).

7.1. Posicionamiento: El Motor de Identidad para la Infraestructura de IA

Thalamus no debe venderse como "Auth0 Open Source en Elixir". Debe posicionarse como **"Infraestructura Crítica de Baja Latencia para la Economía Agéntica"**.

- **Mensaje Clave:** "No permita que la latencia de autenticación frene a sus agentes ni que el costo de tokens destruya su margen de inferencia."

7.2. Hoja de Ruta de Producto Sugerida

Fase 1: El Núcleo de Alto Rendimiento (Q1-Q2 2026)

- **Objetivo:** Paridad funcional con OAuth2 client_credentials y benchmarks públicos irrefutables.
- **Entregable:** Servidor OAuth2 optimizado en Elixir con almacenamiento de tokens en ETS.
- **Métrica Clave:** Demostrar latencia p99 < 5ms bajo una carga de 10k RPS en hardware modesto.
- **Diferenciador:** SDKs que abstraigan la complejidad, permitiendo a desarrolladores de Python (IA) consumir Thalamus sin saber Elixir.

Fase 2: Integración Nativa con el Ecosistema MCP (Q3 2026)

- **Objetivo:** Resolver el problema del "Confused Deputy".
- **Entregable:** Un "Thalamus MCP Gateway". Un servidor proxy que se sienta frente a los servidores MCP existentes (Gmail, Slack) y maneja la autenticación y autorización granular, interceptando y validando scopes antes de que la solicitud llegue a la herramienta.

Fase 3: Liderazgo en Estándares "AAuth" (Q4 2026)

- **Objetivo:** Captura del mercado de cumplimiento y seguridad avanzada.
- **Entregable:** Implementación de referencia del borrador IETF draft-rosenberg-oauth-aauth. Soporte nativo para flujos de consentimiento asíncrono (WebSockets/SSE) donde el agente solicita permiso y "espera" la aprobación humana o

de un supervisor de IA, sin romper la conexión ni hacer polling ineficiente.⁸

8. Conclusión Final y Veredicto

La investigación exhaustiva de las condiciones de mercado a enero de 2026, combinada con un análisis técnico profundo de las limitaciones de la competencia, conduce a un veredicto inequívoco.

Veredicto: GO (PROCEDER)

La convergencia de la demanda masiva de identidades efímeras por la IA Agéntica, sumada a la incapacidad técnica (latencia GC) y económica (precios abusivos) de los proveedores actuales, crea un vacío de mercado perfecto para Thalamus. La elección de la tecnología Elixir/BEAM es el habilitador estratégico que permite transformar este problema en una ventaja competitiva sostenible, ofreciendo un rendimiento superior a una fracción del costo.

Thalamus tiene el potencial de convertirse en el "Nginx de la Identidad Agéntica": una pieza de infraestructura invisible, omnipresente y esencial para la nueva era de la computación autónoma.

Fin del Informe. Preparado por Analista Senior de Infraestructura, Enero 2026.

Fuentes citadas

1. The Security Risks of Model Context Protocol (MCP) - Pillar Security, acceso: enero 16, 2026, <https://www.pillar.security/blog/the-security-risks-of-model-context-protocol-mcp>
2. Security Best Practices - Model Context Protocol, acceso: enero 16, 2026, https://modelcontextprotocol.io/specification/draft/basic/security_best_practices
3. Authorization - Model Context Protocol, acceso: enero 16, 2026, <https://modelcontextprotocol.io/specification/2025-03-26/basic/authorization>
4. OASIS Security: Non Human Identity Management Platform, acceso: enero 16, 2026, <https://www.oasis.security/>
5. Entro Security: Agentic AI & Non-Human Identity Security Platform, acceso: enero 16, 2026, <https://entro.security/>
6. draft-rosenberg-oauth-aauth-00 - IETF Datatracker, acceso: enero 16, 2026, <https://datatracker.ietf.org/doc/html/draft-rosenberg-oauth-aauth-00>
7. draft-patwhite-aauth-00 - AAuth - Agentic Authorization OAuth 2.1 Extension, acceso: enero 16, 2026, <https://datatracker.ietf.org/doc/draft-patwhite-aauth/>
8. AAuth - Agentic Authorization OAuth 2.1 Extension - IETF, acceso: enero 16, 2026, <https://www.ietf.org/archive/id/draft-patwhite-aauth-00.html>
9. Types of Garbage Collectors - Serial | Parallel | CMS GC | G1 GC | - PerfMatrix,

- accesso: enero 16, 2026,
<https://www.perfmatrix.com/types-of-garbage-collectors/>
10. Keycloak Performance Benchmarks: A Deep Dive into Scaling and Sizing (26.4),
accesso: enero 16, 2026, <https://www.keycloak.org/2025/10/keycloak-benchmark>
 11. Keycloak 26.0.7 Performance Challenges for High-Volume IoT Logins #36154 -
GitHub, acceso: enero 16, 2026,
<https://github.com/keycloak/keycloak/discussions/36154>
 12. Keycloak JVM Options, acceso: enero 16, 2026,
https://www.keycloak.org/keycloak-benchmark/kubernetes-guide/latest/running/jvm/jvm_options
 13. JVM ergonomics configuration of Keycloak, and the impact of GC type on
latency - Medium, acceso: enero 16, 2026,
<https://medium.com/@torinks/jvm-ergonomics-configuration-and-gc-type-impact-on-latency-e3f778a85c87>
 14. Too many connections: 1-1 correlation between request concurrency and
connection pool size · keycloak keycloak · Discussion #16122 - GitHub, acceso:
enero 16, 2026, <https://github.com/keycloak/keycloak/discussions/16122>
 15. Concepts for configuring thread pools - Keycloak, acceso: enero 16, 2026,
<https://www.keycloak.org/high-availability/multi-cluster/concepts-threads>
 16. Why is Erlang considered fault tolerant and Java as not in terms of concurrent
programming? - Quora, acceso: enero 16, 2026,
<https://www.quora.com/Why-is-Erlang-considered-fault-tolerant-and-Java-as-not-in-terms-of-concurrent-programming>
 17. Concurrency In The Erlang VM - Skript, acceso: enero 16, 2026,
<https://www.skript.com/blog/concurrency-in-the-erlang-vm>
 18. ETS vs Redis - Anantha Kumaran, acceso: enero 16, 2026,
<https://ananthakumaran.in/2024/10/26/ets-vs-redis.html>
 19. Elixir ETS Versus Redis - CloudBees, acceso: enero 16, 2026,
<https://www.cloudbees.com/blog/elixir-ets-vs-redis>
 20. Unlock Blazing Fast In-Memory Power: The Rails Developer's Guide to Mastering
ETS in Elixir | by Jonny Eberhardt | Medium, acceso: enero 16, 2026,
<https://medium.com/@jonnyeberhardt7/unlock-blazing-fast-in-memory-power-the-rails-developers-guide-to-mastering-ets-in-elixir-4872c3d50909>
 21. Evaluating Elixir / Phoenix for a web-scale, performance-critical application,
accesso: enero 16, 2026,
<https://elixirforum.com/t/evaluating-elixir-phoenix-for-a-web-scale-performance-critical-application/832>
 22. Comparing Elixir vs Java - Erlang Solutions, acceso: enero 16, 2026,
<https://www.erlang-solutions.com/blog/comparing-elixir-vs-java/>
 23. M2M Tokens General Availability - Clerk, acceso: enero 16, 2026,
<https://clerk.com/changelog/2025-10-14-m2m-ga>
 24. Stytc Pricing – The Complete Guide - SuperTokens, acceso: enero 16, 2026,
<https://supertokens.com/blog/stytch-pricing>
 25. 2024.11.22 | Updated Self-Serve Pricing - Changelog - Stytc, acceso: enero 16,
2026,

<https://changelog.stytch.com/announcements/2024-11-22-updated-self-serve-pricing>

26. Auth0 Business Comparison - FusionAuth, acceso: enero 16, 2026, <https://fusionauth.io/resources/fusionauth-vs-auth0.pdf>
27. Auth0 and Okta Enterprise Pricing Explained: The Hidden IDP Tax - FusionAuth, acceso: enero 16, 2026, <https://fusionauth.io/blog/idp-tax>
28. Token Security Achieves Rapid Growth in 2025 as Enterprises, acceso: enero 16, 2026, <https://www.globenewswire.com/news-release/2026/01/14/3218738/0/en/Token-Security-Achieves-Rapid-Growth-in-2025-as-Enterprises-Address-the-Critical-Security-and-Compliance-Risks-of-Agentic-AI.html>
29. Pricing - Auth0, acceso: enero 16, 2026, <https://auth0.com/pricing>
30. 1000 monthly tokens - Auth0 Community, acceso: enero 16, 2026, <https://community.auth0.com/t/1000-monthly-tokens/195808>
31. Securing your app: Using M2M tokens - Clerk, acceso: enero 16, 2026, <https://clerk.com/docs/guides/development/machine-auth/m2m-tokens>
32. FusionAuth Pricing | Free and Flexible Plans for Your Authentication ..., acceso: enero 16, 2026, <https://fusionauth.io/pricing>
33. c7g.2xlarge pricing: \$211.70 monthly - AWS EC2 - Economize Cloud, acceso: enero 16, 2026, <https://www.economize.cloud/resources/aws/pricing/ec2/c7g.2xlarge/>
34. Compare db.m7g.xlarge with db.m7g.large | Pricing, CPU, memory - Aiven, acceso: enero 16, 2026, <https://aiven.io/tools/instances/db.m7g.xlarge-vs-db.m7g.large>